Inventory Management System — Complete API Testing Guide

Base URL: `http://localhost:3000/api`

This guide walks through every module in a sensible order (Items → Customers/Suppliers → Invoices → Payments → Returns → Expenses → Stock Movements → Dashboard), covering normal use, validation edge cases, and cross-table relationship checks.

---

1. Items — `/api/items`

#	Action	Request	Expected Result
1.1	Create item	`POST /items` `{ "code": "ITM001", "name": "Notebook", "purchasePrice": 20, "salePrice": 35, "currentQuantity": 100 }`	`201 Created`, returns item with `itemId`
1.2	Create second item	`POST /items` `{ "code": "ITM002", "name": "Pen", "purchasePrice": 5, "salePrice": 10, "currentQuantity": 200 }`	`201 Created`
1.3	Create low-stock item	`POST /items` `{ "code": "ITM003", "name": "Stapler", "purchasePrice": 40, "salePrice": 60, "currentQuantity": 2 }`	`201 Created`
1.4	Duplicate code	`POST /items` with `code: "ITM001"` again	Rejected — `409 Conflict` or `400`
1.5	Missing required field	`POST /items` without `purchasePrice` / `salePrice`	`400 Bad Request`
1.6	Wrong data type	`POST /items` with `purchasePrice: "abc"`	`400 Bad Request`
1.7	Get all items	`GET /items`	`200`, array of all items
1.8	Get one item	`GET /items/1`	`200`, matches created data
1.9	Get non-existent item	`GET /items/999`	`404 Not Found`
1.10	Update item	`PUT /items/2` `{ "salePrice": 12 }`	`200`, only `salePrice` changes
1.11	Low-stock list	`GET /items/low-stock`	`200`, includes items under threshold
1.12	Movement history (empty)	`GET /items/1/movements`	`200`, empty array before any invoice
1.13	Delete item with no history	Create a throwaway item, then `DELETE /items/:id`	`200` / `204` succeeds
1.14	Search items	`GET /items?search=Pen`	`200`, filtered results by code/name

---

2. Customers — `/api/customers`

#	Action	Request	Expected Result
2.1	Create customer	`POST /customers` `{ "name": "Ahmed Test", "phoneNum": "01012345678" }`	`201 Created`
2.2	Missing required field	`POST /customers` without `name`	`400 Bad Request`
2.3	Duplicate phone number	`POST /customers` with an existing `phoneNum`	Should be rejected — `409 Conflict` (requires unique constraint on `phoneNum`)
2.4	Get all customers	`GET /customers`	`200`, array — each row should include a computed `balance` field
2.5	Get one customer	`GET /customers/1`	`200`
2.6	Get non-existent customer	`GET /customers/999`	`404 Not Found`

2.7	Get balance (no invoices)	`GET /customers/1/balance`	`200`, `balance: 0`
2.8	Update customer	`PUT /customers/1 { "phoneNum": "01000000000" }`	`200`, only phone updates
2.9	Update with Arabic text	`PUT /customers/1 { "name": "أحمد محمد" }`	`200`, Arabic name stored and returned correctly (no corruption)
2.10	Delete with no history	Create throwaway customer, then delete	`200` / `204` succeeds
2.11	Delete blocked when invoices exist	`DELETE /customers/1` (has invoices)	Should be rejected — `400` / `409`, not silently allowed
2.12	Search customers	`GET /customers?search=Ahmed`	`200`, filtered by name/phone

3. Suppliers — `/api/suppliers`

Mirror all of Section 2's tests, using `/suppliers`, `supplierId`, and `purchase` invoices instead of `sale`. Additionally:

#	Action	Request	Expected Result
3.1	Create with `type` field	`POST /suppliers { "name": "Test Supplier", "phoneNum": "...", "type": "factory" }`	`201`, `type` stored and returned (requires `type` column in schema)
3.2	Get balance (no invoices)	`GET /suppliers/1/balance`	`200`, `balance: 0`

4. Invoices — `/api/invoices`

#	Action	Request	Expected Result
4.1	Create sale invoice	`POST /invoices { "invoiceNum": "INV-1001", "invoiceType": "sale", "customerId": 1, "invoiceDate": "...", "terms": [{ "itemId": 1, "quantity": 5, "price": 35 }] }`	`201`, transactional — creates Invoice + Invoice_Terms + Stock_Movement rows
4.2	Verify stock decreased	`GET /items/:id` for each item sold	`currentQuantity` reduced by sold amount
4.3	Verify stock movement created	`GET /items/:id/movements`	New row: `movementType: "sale_out"`, correct `quantity`, `invoiceId` set, `returnId` null
4.4	Get invoice with totals	`GET /invoices/:id`	Includes derived total, amount paid, return total, and outstanding balance
4.5	Get invoice list	`GET /invoices`	`200`, each row should include total/paid/remaining/status (not just raw fields)
4.6	Filter by type	`GET /invoices?invoiceType=sale`	Only sale invoices returned
4.7	Filter by invoice number	`GET /invoices?invoiceNum=INV-1001`	Returns matching invoice only
4.8	Filter by date range	`GET /invoices?from=2026-08-01&to=2026-08-31`	Includes invoices within range
4.9	Create purchase invoice	`POST /invoices` with `invoiceType: "purchase"`, `supplierId`, no `customerId`	`201`, stock **increases** afterward, movement type `purchase_in`
4.10	Both customer + supplier filled	`POST /invoices` with both IDs set	`400 Bad Request` — exactly one allowed
4.11	Neither filled	`POST /invoices` with neither ID set	`400 Bad Request`
4.12	Invalid invoice type	`POST /invoices` with `invoiceType: "rental"`	`400 Bad Request`
4.13	Duplicate invoice number	`POST /invoices` reusing an existing `invoiceNum`	Rejected — unique constraint
4.14	Attempt edit	`PUT /invoices/:id`	Confirm whether implemented; if not, `404` is expected by design

4.15	Attempt delete (has activity)	`DELETE /invoices/:id`	Should fail if the route exists, since Stock_Movement rows reference it (foreign key protection)

---

5. Payments — `/api/payments`

#	Action	Request	Expected Result
5.1	Register payment	`POST /payments` `{ "invoiceId": 1, "amount": 50, "paymentDate": "...", "paymentMethod": "cash" }`	`201 Created`
5.2	Verify invoice updated	`GET /invoices/1`	Amount paid and outstanding balance reflect the payment
5.3	Verify customer balance updated	`GET /customers/1/balance`	`totalPayments` and `balance` reflect the payment
5.4	Overpayment rejection	`POST /payments` with amount greater than outstanding balance	`400 Bad Request` — “Payment exceeds remaining balance”
5.5	Get all payments	`GET /payments`	`200` , array including your payment
5.6	Get one payment	`GET /payments/:id`	`200` , matches created data
5.7	Update payment	`PUT /payments/:id` `{ "amount": 60 }`	`200` (currently unrestricted by design)
5.8	Re-check invoice after edit	`GET /invoices/1`	Amount paid reflects the new value
5.9	Delete payment	`DELETE /payments/:id`	`200` / `204`
5.10	Re-check invoice after delete	`GET /invoices/1`	Amount paid reverts, outstanding balance restored — confirms live (not cached) calculation
5.11	Add payment with notes	`POST /payments` including `"notes": "دفعة أولى"`	`201` , Arabic notes stored and returned correctly

---

6. Returns — `/api/returns`

#	Action	Request	Expected Result
6.1	Create return against sale invoice	`POST /returns` `{ "invoiceId": 1, "returnType": "from_customer", "customerId": 1, "reason": "...", "items": [{ "itemId": 2, "quantity": 2, "price": 12 }] }`	`201 Created`
6.2	Verify stock increased	`GET /items/2`	`currentQuantity` increased by returned amount
6.3	Verify stock movement	`GET /items/2/movements`	New row: `movementType: "sale_return_in"` , `returnId` set, `invoiceId` null
6.4	Verify invoice totals updated	`GET /invoices/1`	Return total factored into outstanding balance
6.5	Get return by id	`GET /returns/:id`	Includes derived return total and `reason`
6.6	Get returns list	`GET /returns`	Each row includes derived return total, not just raw fields
6.7	Filter by invoice	`GET /returns?invoiceId=1`	Only returns for that invoice
6.8	Filter by type	`GET /returns?returnType=from_customer`	Only matching type
6.9	Mismatched return type vs invoice type	Return `to_supplier` against a sale invoice	`400 Bad Request`
6.10	Both/neither customer+supplier	Same “exactly one” rule as Invoices	`400 Bad Request`

6.11	Over-return quantity	Return more than was originally invoiced	`400 Bad Request`
6.12	Create return against purchase invoice	`returnType: "to_supplier"`	`201`, stock **decreases**, movement type `purchase_return_out`

---

7. Expenses — `/api/expenses`

#	Action	Request	Expected Result
7.1	Create expense	`POST /expenses` `{ "type": "إيجار", "amount": 8000, "description": "..." }`	`201 Created`
7.2	Missing required field	`POST /expenses` without `type`	`400 Bad Request`
7.3	Get all expenses	`GET /expenses`	`200`, array with expense date auto-set
7.4	Search expenses	`GET /expenses?search=إيجار`	Filtered by type/description
7.5	Update expense	`PUT /expenses/:id`	`200`
7.6	Delete expense	`DELETE /expenses/:id`	`200` / `204`

---

8. Stock Movements — `/api/stock-movements` (read-only)

#	Action	Request	Expected Result
8.1	Get all movements	`GET /stock-movements`	`200`, includes item code/name, movement type, quantity, reference IDs, date
8.2	Search by item	`GET /stock-movements?search=Pen`	Only movements involving that item
8.3	Filter by date range	`GET /stock-movements?from=...&to=...`	Only movements within range
8.4	Confirm no create endpoint exists	`POST /stock-movements`	`404` — movements are only ever created as a side-effect of Invoices/Returns

---

9. Dashboard — `/api/dashboard`

#	Action	Request	Expected Result
9.1	Get summary	`GET /dashboard/summary`	`200`, returns today's sales, current stock value, low-stock count, total receivable
9.2	Verify today's sales	Cross-check against invoices created today	Matches sum of today's sale invoice totals
9.3	Verify stock value	Cross-check against items	Matches sum of (quantity × purchase price) across all items
9.4	Verify total receivable	Cross-check against customer balances	Matches sum of all customer balances
9.5	Recent invoices	`GET /invoices` (reused, sorted newest-first)	Frontend takes first 3 entries for the dashboard's recent-invoices section

---

10. Cross-Module Relationship Checks

These confirm the automation chain works end-to-end, not just each module in isolation.

#	Scenario	Steps	Expected Result
10.1	Sale reduces stock	Create sale invoice → check item quantity	Quantity decreases by sold amount

10.2	Purchase increases stock	Create purchase invoice → check item quantity	Quantity increases by purchased amount
10.3	Return reverses a sale	Create return on a sale invoice → check item quantity	Quantity increases back
10.4	Return reverses a purchase	Create return on a purchase invoice → check item quantity	Quantity decreases back
10.5	Payment reduces outstanding balance	Add payment → check invoice and customer balance	Both reflect the reduced balance
10.6	Deleting a payment restores balance	Delete payment → check invoice and customer balance	Both revert to pre-payment values
10.7	Stock movement always traces to exactly one source	Inspect any Stock_Movement row	Exactly one of `invoiceId` / `returnId` is set, never both, never neither
10.8	Customer balance matches sum of their invoices	Compare `GET /customers/:id/balance` against manually summing their invoices, returns, and payments	Numbers match exactly
10.9	Supplier balance matches sum of their invoices	Same as above, for purchase invoices	Numbers match exactly

---

Notes on Known Design Decisions

- All totals and balances (invoice total, paid amount, outstanding balance, customer/supplier balance, return total) are **calculated live on every request**, never stored as cached fields — this guarantees the numbers can never drift out of sync with the underlying data.
- Stock_Movement rows are **never created directly by the user** — they only ever appear as an automatic result of creating an Invoice or a Return.
- Invoices are effectively **permanent once they have activity** (line items, payments, or stock movements) — they cannot be hard-deleted, by design, to preserve the audit trail.